Software Engineering Assignment 04

Smart Campus Room Availability Checker

Boranov Daniyar

Student ID 24B031718

Thursday 14:00–17:00

1 Project overview and prototype access

The Smart Campus prototype now supports four fictional university rooms. Capacity and equipment filters help users choose a suitable room. The availability check then uses only that room's blocked status and existing bookings. The original time rules and read-only booking behavior are preserved.

Live A4 demonstration https://smart-campus-room-av-j4pb.bolt.host/

The A4 application was synchronized with the existing Bolt.new project and published at the original Bolt.host address. All 22 browser checks passed without signing in. The published JavaScript and CSS match the tested local production build byte for byte.

Existing Bolt project https://bolt.new/~/sb1-bskzikuu

2 Original implementation and recoverable source

The original A3 implementation is preserved at commit 55cab2a186df4107ab7189f509c259006af28f3f. The annotated tag a4-baseline-2026-11-05 and the verified recovery bundle keep the starting version recoverable. The earlier prototype-v1 tag and historical A3 evidence are also unchanged.

Final A4 source commit 10eeb3812d6f9491cd47dcd72f5bf03b0884f88c. The package contains source references, test files, command logs and recovery instructions in README.md.

Scope	Implementation
Original rules	Integer minutes, fixed now 540, maximum 120 minutes, blocked rejection, touching endpoints allowed
New behavior	Four rooms; capacity and ALL-equipment filters; empty state; room switching
Data boundary	Fictional data only; no booking creation or changes

3 Codebase map and maintainability risks

The analysis used real source and function references from the preserved baseline. The risks affect the extension of a working one-room application.

Source	Responsibility
src/App.tsx App and handleCheck	Application state, checking and component composition
src/types.ts and data/sampleData.ts	Original domain types, A-102, fixed time and examples
src/utils/availability.ts checkAvailability	Pure AC1–AC5 validation and booking overlap checks
src/components	Time controls, room details, bookings and result feedback
scripts/frontend-test.mjs	Actual TypeScript availability function tested with 37 fixtures
booking.py can_book and test_booking.py	Python reference implementation and 47 tests

Risk	Baseline evidence	Change
Global booking coupling	App.tsx lines 27–35 and 95 use SAMPLE_BOOKINGS.	Check and display derive data from the selected room.
Small room model	types.ts lines 8–12 lacks capacity, equipment and bookings.	Readonly room catalog stores all required fields.
Repeated invalidation	Five handlers clear results at App.tsx lines 42, 47, 52, 57 and 64.	One state transition path invalidates prior results.
No executable UI events	frontend-test.mjs lines 17–49 tests pure rules only.	Real Chromium tests exercise filtering and switching.
Repeated feedback wrapper	ResultPanel.tsx has three repeated status sections.	A shared accessible wrapper presents conditional feedback.

4 Prioritized improvement plan

P1 added the room model and fictional catalog. P2 extracted pure filtering and connected the selected room to checking and booking display. P3 added the controls, selection handling and shared feedback. P4 ran the final regression and UI checks and collected evidence. Each step kept changes small enough to inspect in the Git diff.

5 Preservation of AC1 to AC6

The availability function retains the original validation contract. Room selection changes its input data, while the meaning of valid and invalid requests stays the same.

Criterion	Preserved behavior	Source and verification
AC1	Whole minutes; 0 ≤ start < end ≤ 1440; start > 540.	utils/availability.ts checkAvailability; invalid time and day-boundary fixtures
AC2	Exactly 120 minutes allowed; 121 rejected.	Original duration guard and separate regression/UI boundary checks
AC3	A blocked room cannot be available.	Existing blocked guard; catalog-blocked room and UI checks
AC4	Overlaps rejected; touching endpoints allowed.	start < booking.end AND end > booking.start; boundary fixtures
AC5	Only check availability; bookings never change.	Readonly/frozen catalog and deep equality checks
AC6	Clear result/reason and responsive controls.	Shared ResultPanel, labelled controls and narrow-screen browser tests

6 New room catalog

Room and capacity	Equipment	Blocked	Bookings
A-102 Study Room A-102 4 people	Whiteboard, projector	No	10:00–11:00
B-201 Team Room B-201 8 people	Whiteboard, display	No	11:00–12:00
C-303 Seminar Room C-303 12 people	Projector, display, video conference	No	12:00–13:00
D-404 Project Room D-404 20 people	Whiteboard, projector, video conference	Yes	14:00–15:00

Each room has an ID, name, capacity, equipment list, blocked status and its own immutable bookings. A-102 keeps the original booking from 600 to 660 minutes. The current demo time remains 540 minutes, or 09:00.

7 Filtering and room selection

Rule	Final behavior	Verification focus
F1	Capacity must be greater than or equal to attendees.	Exact, insufficient and larger capacity
F2	Every required equipment item must be present. An empty selection adds no equipment restriction.	ALL matching, partial match and no equipment
F3	A matching blocked room remains clearly marked and cannot become Available.	D-404 blocked guard and UI
F4	No matching rooms produces a clear empty state. Old feedback and invalid selection are cleared.	Zero-match browser event
F5	Room switching clears prior feedback; checking uses the new room's status and bookings.	A-102 available versus B-201 overlap for 11:00–12:00

Capacity and equipment conditions are combined with AND. Attendee count must be a positive whole number. Selecting a room stores its ID and derives the current room record. If new filters exclude that room, selection and feedback are cleared, and checking is disabled until a visible room is selected.

The original example cases are scoped to A-102. They cannot silently check a room hidden by the current filters. Bookings remain read-only throughout filtering, switching and checking.

8 Refactoring design decisions

Room matching and availability are separate pure functions. This keeps capacity and equipment rules easy to test while preserving the original time and overlap validator. The trade-off is that the interface must explain that a matching room can still be blocked or have a booking conflict.

Selection uses a room ID rather than a second copy of room data. The booking list and availability check therefore use one derived record. A common state update path clears obsolete results. This introduces explicit state transitions, but makes F5 behavior testable without duplicating the validator.

The result panel uses one shared status wrapper. Available, unavailable and initial content keep different messages within the same labelled live region. This reduces repeated rendering code and keeps accessibility attributes consistent.

9 Module and component diagrams

The BEFORE diagram reflects the original one-room source. The AFTER diagram reflects the final catalog, filter, state and presentation modules. Diagram source files and readable images are included in diagrams/.

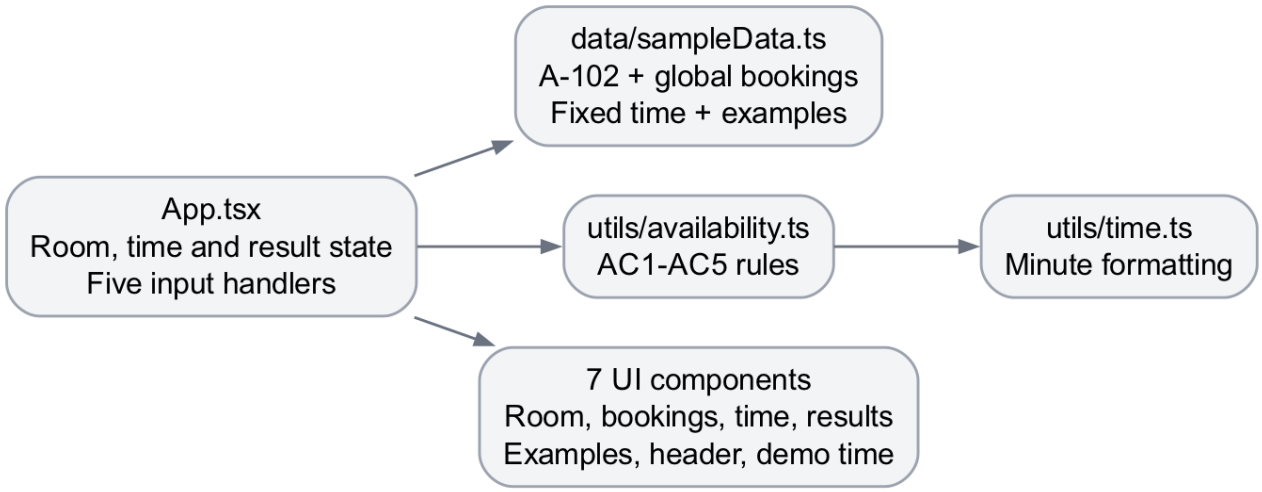


Figure 1 Original A3 module and component structure

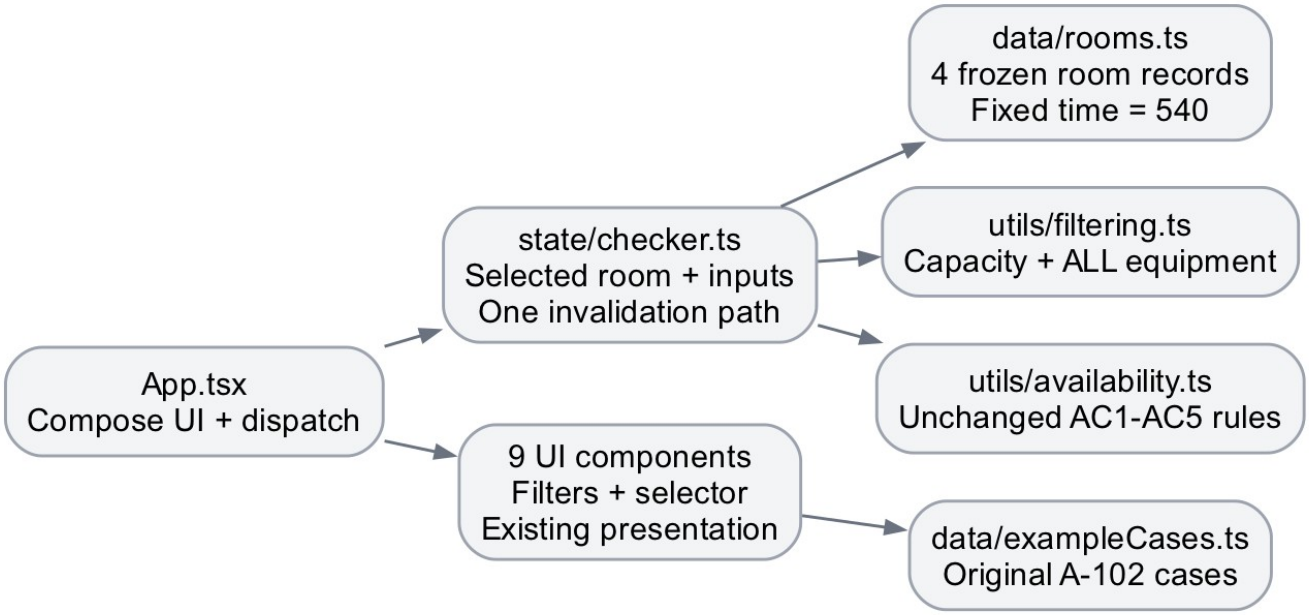


Figure 2 Final A4 module and component structure

10 Automated testing and execution results

The existing Python and frontend regression tests were retained. New tests use the production catalog, filtering functions and state transitions. Chromium tests execute real interface events, including room switching, filtering, empty state and mobile layout.

Check	Actual result	Executed command
Python regression	47/47 passed	python3 -m unittest -v test_booking
Frontend rules and catalog/state	76/76 passed	npm test
ESLint	Exit 0	npm run lint
TypeScript	Exit 0	npm run typecheck
Production build	Exit 0	npm run build -- --outDir output/a4-final-build
Chromium integration and UI	22/22 passed	A4_UI_RUN=final-local npm run test:ui -- --project=chromium
Anonymous public Chromium UI	22/22 passed	A4_BASE_URL=https://smart-campus-room-av-j4pb.bolt.host/ A4_UI_RUN=public-bolt-final npm run test:ui -- --project=chromium

The original baseline passed Python 47/47 and frontend 37/37. These results were rerun and saved before source changes. Final counts come from the test runners, not from counting assertions or screenshots. Full commands, stdout/stderr, exit codes and reports are included in evidence/.

11 Test coverage and evidence

Coverage	Relevant behavior
AC1–AC4 regression	Invalid/equal/reversed times; 120/121 minutes; blocked room; touching and overlapping intervals; multiple bookings
AC5 and room isolation	Frozen booking inputs unchanged; each room uses its own booking list
F1 and F2	Exact and insufficient capacity; ALL equipment; missing item; empty equipment selection
F3–F5 UI	Blocked room; no matches; switching rooms; filters invalidating selection; no stale result
AC6 UI	Labelled controls, clear feedback and narrow/mobile layout

A normal verification build was changed to use an ignored output directory because dist/ already contains tracked deployment files. This preserves the previous snapshot during checks. Publication status is verified separately from local test results.

12 Filtering and availability evidence

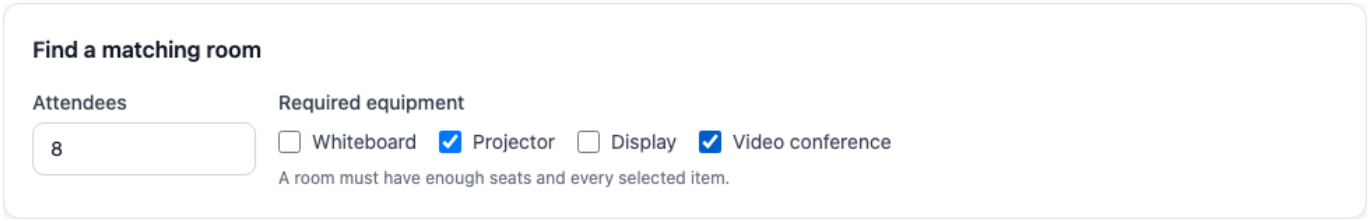


Figure 3 Capacity and equipment requirements. Eight attendees, projector and video conference are required. Capacity and ALL equipment conditions apply together.

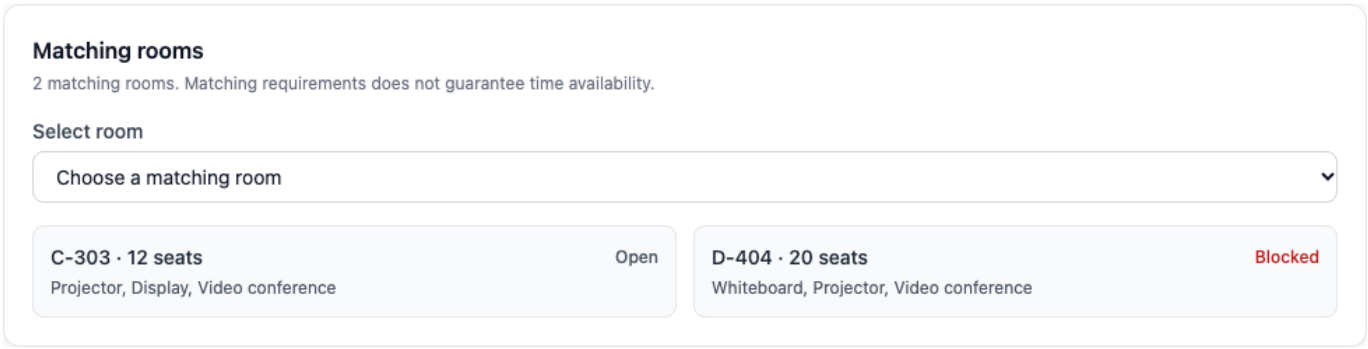


Figure 4 Only matching rooms are displayed. Only C-303 and D-404 satisfy the selected requirements; D-404 remains clearly blocked.

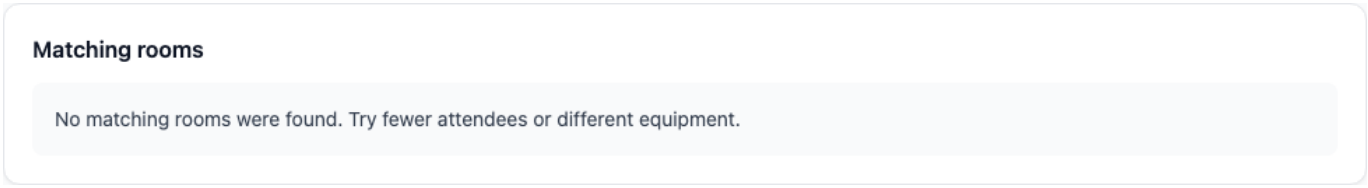


Figure 5 Clear empty state when no rooms match. Thirteen attendees plus Display have no matching room. Selection and previous feedback are cleared; checking is disabled.



Figure 6 The same 11:00–12:00 request is available in C-303 and unavailable in B-201. Each result uses that room's own bookings.

These are complete semantic-element captures from real browser checks. Their capture records, viewport sizes and original screenshots are included in the evidence folder.

13 Selected room and responsive evidence



Figure 7 The selected C-303 record supplies its own capacity, equipment and status.

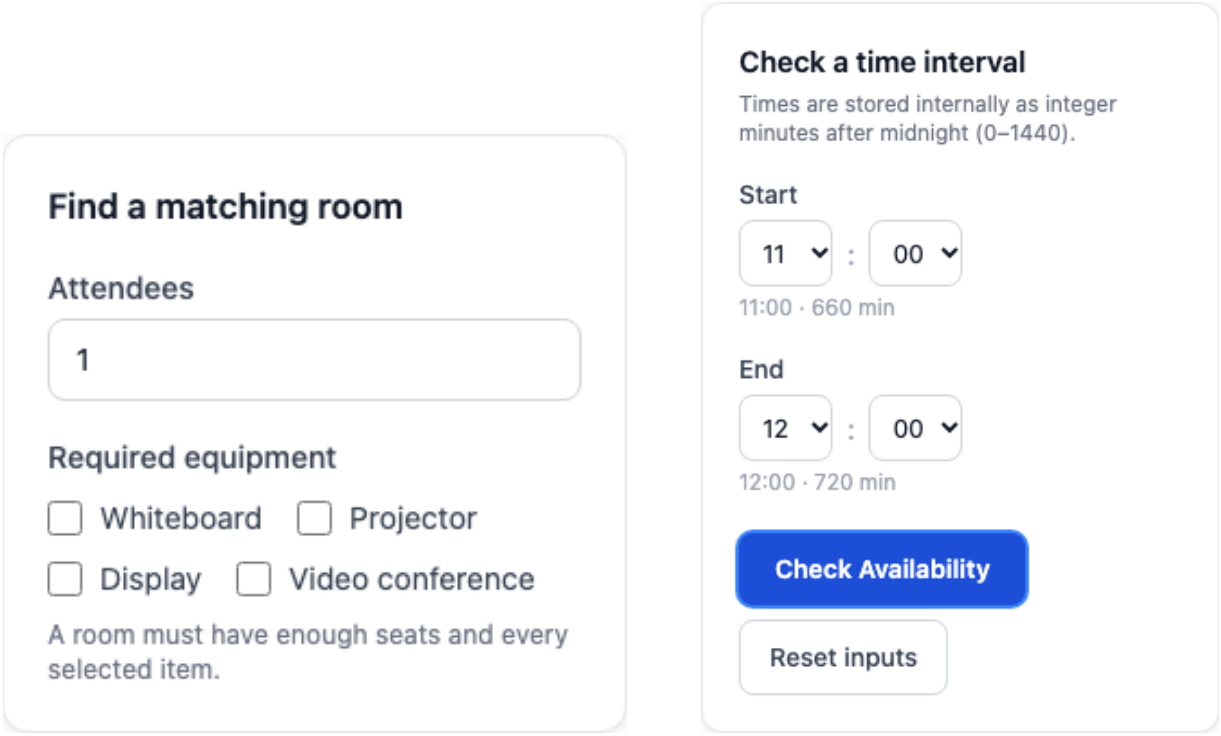


Figure 8 At a 320-pixel viewport, the complete requirements and original time controls remain readable and usable. Both semantic-element screenshots are displayed without cropping.

Changing rooms removes the previous availability feedback. A later check uses the chosen room's existing bookings. On a narrow screen the controls remain usable without horizontal scrolling. The screenshots and UI assertions cover these behaviors separately.

14 Measured quality evaluation

Measure	A3 baseline	Final A4	How measured
Python regression	47/47	47/47	unittest runner totals in saved logs
Frontend logic	37/37	76/76	Node test runner totals; existing fixtures retained
Executable UI coverage	No committed executable UI suite	22/22	Chromium test report, excluding screenshots from test count
Repeated status wrappers	3	1	ResultPanel source before/after; repeated wrapper locations removed
Result invalidation locations	5	1	Input handlers before versus common transition after
Production modules	13	18	TS/TSX production files, excluding declarations; saved file lists
UI components	7	9	Source files under src/components; module responsibilities shown in diagrams

Two repeated status-wrapper locations were removed (3 to 1), and four repeated input-result reset locations were removed (5 to 1). The final locations are ResultPanel.tsx and state/checker.ts updateInputs. The larger catalog and filtering controls are feature additions. Pure filtering, one selected-room state path and shared status presentation improve maintainability. Duplication measurements count active production locations only; preserved A3 source copies are not removed or counted as refactoring gains.

15 Remaining limitations

Fictional room data and fixed 09:00 time; no real reservations or persistence.

Chromium automation covers responsive layout and keyboard interactions; a full screen-reader and cross-browser audit remains outside this verification.

The current dependency audit reports 12 advisories (2 low, 9 moderate, 1 high, 0 critical); no unrelated dependency upgrade was applied.

16 Reflection

For Assignment 04, I chose to extend the existing Smart Campus prototype. I provided the previous source, the original acceptance criteria, the assignment PDF and clear limits. I approved a small plan that kept the current framework, fixed demo time and booking rules.

The most useful AI suggestion was to separate room matching from availability checking. This made the new filters easier to understand and test. Keeping the selected room ID also reduced the risk of showing one room while checking another room's bookings. A shared result panel made the status messages more consistent.

One recommendation was modified for a technical reason. A normal production build would overwrite the tracked deployment folder. The verification build therefore used a separate ignored output folder. I also kept the original overlap comparison because touching booking endpoints must remain valid. A backend, live clock and booking creation were outside the agreed scope.

The verification evidence includes real Python and frontend regression results, new filtering and state tests, and browser checks. Room switching and empty-state tests were especially useful. A correct validation function alone cannot prove that the interface passes the right room data. These tests supported the decision to clear old results whenever the room or requirements change. One keyboard check initially used an unreliable key sequence. It was corrected to native type-ahead after a focused browser check; the application did not need a change.

I would next improve keyboard and screen-reader testing. A larger catalog would also need clearer data validation and more equipment combinations. For this assignment, a small fictional catalog keeps the behavior reproducible and the design easy to inspect.

17 AI use and references

Codex supported codebase analysis, implementation, test design and documentation. Bolt.new was used to synchronize and publish the existing project. Recommendations were checked against the assignment, source code, Git diffs and actual test output. The separate AI evidence appendix records prompts, outputs, review decisions and verification sources.

[1] Software Engineering Assignment 04, supplied 30-page PDF, especially pages 10–19 and 25–27.

[2] Smart Campus Room Availability Checker, preserved A3 baseline 55cab2a186df4107ab7189f509c259006af28f3f and final A4 source 10eeb3812d6f9491cd47dcd72f5bf03b0884f88c.

[3] Playwright documentation, Installation and Web server, used for the executable Chromium checks.